

SmartCare-AI

Deployment, Migration and Audit Backup Notes

This document summarizes how the current SmartCare-AI development installation can later be moved to a client's hosting environment, domain and database, and explains the production-hardening note about keeping audit backups independently from the application server.

1. What does "off-server backup" mean?

At the moment, the SmartCare-AI application/database environment, the audit checkpoint files, and the checkpoint signing key are associated with the same development machine. The Windows permissions configured during F13 restrict ordinary access, but they do not make that machine an independent trust boundary.

If the whole machine is lost, damaged, compromised by ransomware, or controlled by an administrator, the database and its local protection material could potentially be affected together.

Off-server backup means retaining protected copies outside the application server - for example, in separately controlled backup storage. In a production deployment, database backups and audit evidence should ultimately have an independent recovery location and appropriate access controls.

This is a production-hardening task. It does not mean the current F13 implementation is unfinished.

2. Will SmartCare-AI need to be rewritten for every client?

No. The intended deployment model is one SmartCare-AI codebase with installation-specific configuration. Moving to a different server or domain should normally involve deployment, configuration, testing and, where necessary, data migration - not rewriting the application.

3. What normally stays the same?

- Laravel backend and API logic
- React frontend and workflows
- Database models and migrations
- Audit logging and integrity verification logic
- Automated tests

4. What changes for each installation?

- Domain name and HTTPS configuration
- Database host, database name, username and password
- Application URL and frontend API address
- Audit checkpoint directory
- Audit signing-key file location
- Server permissions, scheduled jobs and backup arrangements

For F13 specifically, the checkpoint paths were moved into environment-specific configuration. This means a client's server can use different paths without changing the checkpoint service's PHP logic.

5. Database migration scenarios

Scenario	Typical requirement
New empty MySQL database	Configure client credentials, run migrations, and initialize required accounts/data.
Move existing SmartCare-AI data	Perform a controlled database migration and verify audit records and checkpoints.
Different database engine	Perform compatibility testing and adapt database-specific behavior if necessary.

Important F13 rule: a fresh client installation should have its own signing key and initial checkpoint. If existing audit history is transferred, its corresponding signing key and retained checkpoint files must also be preserved; replacing the key would prevent verification of those existing signatures.

6. What could make deployment harder?

The main deployment risk is not simply changing a domain name or database credentials. The important task is identifying machine-specific assumptions elsewhere in the application, such as hardcoded localhost URLs, Windows-only paths, local storage assumptions, XAMPP dependencies, server-specific permissions, or environment-specific background processes.

F13's checkpoint locations have already been made configurable, but a complete deployment-readiness review of the whole SmartCare-AI project should still be performed before production delivery.

7. Recommended deployment approach

Complete the remaining development phases, then perform a deployment-readiness review and test the application on a clean server environment. This provides an opportunity to find configuration assumptions before deploying to a client's production system.

Bottom line

SmartCare-AI should remain one application, not a different codebase for every client. A new client should normally require installation, environment configuration, database setup or migration, security configuration, and deployment testing. The goal is to avoid rewriting application code simply because the client uses a different server, database instance or domain.